Differential Evolution: Study of Real-Valued Continuous Crossover Operators

J.M. Ward
Computer Science Department
Stellenbosch University
Stellenbosch, South Africa
24913707@sun.ac.za

***Abstract*—Differential Evolution (DE) algorithms have been developed to solve continuous-valued optimisation problems. Interestingly, the canonical DE algorithm utilises discrete recombination crossover to build child vectors throughout the algorithm's life span. In this report advanced continuous crossover operators are implemented and tested against 8 multi-modal benchmark functions to determine if crossover mechanisms intentionally designed for real-valued representations outperform standard discrete recombination. By specifically implementing blending crossover operators and using them in DE algorithms, a significant improvement in continuous optimisation performance and accuracy is hypothesised. Experimental evaluation was performed at 30 dimensions over 30 independent runs for 3000 generations each. Results indicate that continuous blending methods, particularly those utilising expanded barycentric and normal distributions, significantly outperform the discrete baseline on macro-gradient and deceptive landscapes. However, exploitative parent-centric methods remain vulnerable to premature convergence.**

I. Introduction

Differential Evolution (DE) is a population-based stochastic optimisation algorithm widely utilised for solving complex, non-linear continuous optimisation problems. The algorithm improves a candidate population iterativley through mutation, recombination and selection. While DE has proven highly successful, its canonical formulation contains a fundamental geometric contradiction when applied to continuous landscapes.

The standard DE algorithm-such as the widely used DE/rand/2/bin varient-relies on discrete binomial recombination to generate child vectors. Binomial crossover operates by independently swapping exact parameter values between a target vector and a mutated trial vector. While computationally efficient, this discrete swapping mechanism restricts the algorithm's search trajectory to a rigid grid, which limits its ability to effectively navigate continuous, non separable landscapes.

To overcome the geometric limitations of discrete recombination, real-valued blending crossover operators have been developed. Operators such as Uniform Arithmetic Crossover (AX), Simplex Crossover (SPX), Unimodal Normal Distribution Crossover (UNDX), and Parent-Centric Crossover (PCX), generate offspring within a continuous mathematical space, such as barycentric expanded triangles or multi-dimensional Guassian ellipsoids. Because these methods generate new vector geometry defined by the relative positions of the parent vectors rather than the axes of the search grid, they are coordinate-independent and offer highly specialised mechanisms for both local exploitation and global exploration.

The primary goal of this report is to investigate the efficacy of integrating these continuous blending crossover operators into the standard DE algorithm. By replacing the canonical binomial crossover with AX, SPX, UNDX and PCX, this study evaluates whether implemented real valued recombination operators outperform discrete parameter swapping. The modified algorithms were benchmarked across 8 multi-modal, 30 dimensional functions. Experimental observations confirm continuous blending crossover operators significantly enhance performance on specific topographies. Crossover operators AX and UNDX demonstrated superior exploitation on macro-gradient landscapes, while the expanded exploratory geometry of PSX successfully bypassed deceptive geometric traps. Conversely, highly exploitative operators like PCX were observed to be uniquely vulnerable to premature convergence on multi-modal surfaces.

The remainder of this report is organised as follows. Section 2 details the mathematical formulations of the implemented crossover operators. Section 3 outlines the experimental setup, benchmark functions and testing parameters. Section 4 present the convergence data and provides comparative analysis of the algorithms' performances. Finally, Section 5 concludes the report and summarises the key findings.

II. Background

Continuous optimisation involves finding the global minimum or maximum of an objective function within a multi-dimensional search space. These geometric landscapes are multi-modal and can contain many local optima that act as traps for traditional gradient-based search methods. To overcome this challenge, stochastic metaheuristic algorithms are deployed to the landscapes. These algorithms must balance exploration (searching broadly across the entire landscape) with exploitation (narrowing the search locally to fine tune a solution) in order to successfully converge on the true global solution.

Differential Evolution (DE) is a prominent subset of Evolutionary Algorithms (EA) explicitly developed to solve continuous-valued optimisation problems. Like traditional EA's, DE utilises a population of candidate solutions that evolve over successive generations. However, unlike other EA's that apply selection before mutation, DE performs the mutation

of a random vector before applying selection. The mutation process itself also differs in the manner that mutation step sizes are not sampled from a prior known probability distribution function but rather, the mutation step size is influenced by the differences between individuals of the current population.

The canonical DE algorithm, frequently denoted by the mutation and crossover strategy DE/rand/1/bin, operates through a continuous loop of four primary phases: Initialisation, Mutation, Recombination, and Selection. During mutation, a trial vector is created by adding together a weighted difference of randomly selected population vectors to the target vector. Recombination then mixes the parameters of the trial vector and the target vector to create offspring vectors. Finally an elitist selection mechanism compares the target vector to the spawned offspring vector, keeping strictly the candidate that yields the better fitness. The high level execution of a standard DE algorithm is illustrated in Algorithm 1.

Algorithm 1 Standard Differential Evolution (DE)

Initialize population $P = \{\mathbf{x}_1, \ldots, \mathbf{x}_{NP}\}$ randomly within search bounds
while termination condition is not met **do**
  for $i = 1$ to NP **do**
    // Mutation
    Select distinct random indices from the population, excluding i
    Generate a trial vector $\mathbf{v}_i$ using vector differences and scale factor F
    // Recombination
    Generate an offspring vector $\mathbf{u}_i$ by recombining the trial vector $\mathbf{v}_i$ and target $\mathbf{x}_i$ based on crossover rate CR
    // Selection
    Evaluate the fitness of the offspring $f(\mathbf{u}_i)$
    if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ **then**
      $\mathbf{x}_i^{next} \leftarrow \mathbf{u}_i$
    else
      $\mathbf{x}_i^{next} \leftarrow \mathbf{x}_i$
    end if
  end for
  $P \leftarrow P^{next}$
end while

The recombnation phase is critical for injecting diversity and guiding the search geometry. In standard DE this is achieved using discrete operators like binomial or exponential crossover. These algorithms build offspring vectors by independently swapping exact parameter values along the search axes. This approach is coordinate dependent and struggles on rotated or non-separable landscapes. To address this, extensive literature within the domain of Real-Coded Genetic Algorithms (RCGAs) has explored continuous blending operators. Instead of swapping axis coordinates, these continuous operators create offspring by exploring the mathematical volume or geometry between parents. By utilizing techniques like arithmetic interpolation, simplex expansions, or Gaussian probability distributions, the geometric space between target vectors can be explored in much finer detial.

III. Methodology

To investigate the performance differences between discrete and continuous blending recombination, a canonical DE/rand/2 Differential Evolution framework was implemented in Python. The standard binomial crossover was established as the baseline solver. Subsequently, four distinct continuous blending crossover operators were implemented by exclusively modifying the recombination phase of the algorithm. The mutation and selection methods were left constant.

A. Baseline: Binomial Crossover (DE/bin)

The control algorithm utilized the standard discrete binomial crossover. For a given target vector $\mathbf{x}_i(t)$ and a mutant donor vector $\mathbf{v}_i(t)$, the trial vector $\mathbf{u}_i(t)$ was constructed by independently evaluating a uniform random number against the crossover rate (CR) for each dimension j. To prevent stagnation, a randomly selected index j_{rand} was guaranteed to inherit from the donor vector.

Algorithm 2 Binomial Crossover (DE/bin)

Require: Target vector $\mathbf{x}_i$, Trial vector $\mathbf{v}_i$, Crossover Rate CR, Dimension D
Ensure: Off spring vector $\mathbf{u}_i$

$j_{rand} \leftarrow$ random integer in $[1, D]$
for $j = 1$ to D **do**
  $r \leftarrow$ random uniform number in $[0, 1]$
  if $r < CR$ **or** $j = j_{rand}$ **then**
    $u_{i,j} \leftarrow v_{i,j}$ *// Inherit from trial vector*
  else
    $u_{i,j} \leftarrow x_{i,j}$ *// Inherit from target*
  end if
end for
return $\mathbf{u}_i$

B. Uniform Arithmetic Crossover (DE/AX)

To implement a highly explorative continuous blend, a Uniform Arithmetic operator was formulated. Unlike standard linear arithmetic crossover which applies a single scalar across all dimensions, this implementation decoupled the blending weights. For every dimension j, an independent random scalar $\gamma_j \in [0, 1]$ was sampled:

$$u_{ij}(t) = (1 - \gamma_j)x_{ij}(t) + \gamma_j v_{ij}(t) \tag{1}$$

Geometrically, this guarantees the offspring are sampled uniformly from the multi-dimensional hyper-rectangle bounded by the target and donor vectors.

Algorithm 3 Uniform Arithmetic Crossover (DE/AX)

Require: Target vector $\mathbf{x}_i$, Trial vector vector $\mathbf{v}_i$, Dimension D

Ensure: Offspring vector $\mathbf{u}_i$

for $j = 1$ to D **do**
  $\gamma_j \leftarrow$ random uniform number in $[0, 1]$
  $u_{i,j} \leftarrow (1 - \gamma_j) \cdot x_{i,j} + \gamma_j \cdot v_{i,j}$
end for
return $\mathbf{u}_i$

C. Simplex Crossover (DE/SPX)

The Simplex operator was implemented using $m = 3$ parents: the target vector, the donor vector, and a randomly selected third vector from the population. The center of mass $\mathbf{O}$ of these three vertices was calculated, and the simplex was expanded outward using an expansion rate of $\epsilon = \sqrt{3}$. Offspring were uniformly sampled from within this expanded barycentric geometry by generating two random weights (r_1, r_2), shown in Figure 1. A fold trick was applied (`if` $r_1 + r_2 > 1.0$: $r_1 = 1 - r_1$; $r_2 = 1 - r_2$) to ensure uniform interior distribution without skewing.

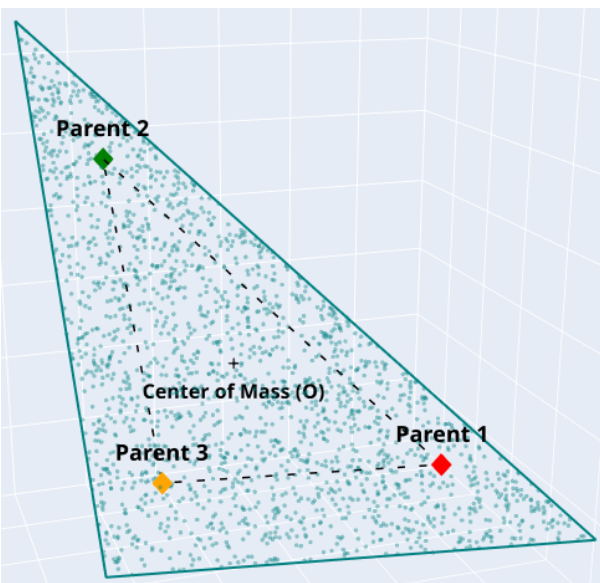

Fig. 1. Geometric visualization of the SPX operator, demonstrating outward simplex expansion around the center of mass $\mathbf{O}$ and uniform sampling of offspring within the expanded triangular bounds.

D. Unimodal Normal Distribution Crossover (DE/UNDX)

The classic 3-parent UNDX operator was implemented to search the probability space between solutions, as visualized in Figure 2. The normal distribution was centered exactly at the midpoint $\mathbf{m}$ between the target and donor vectors, with the primary axis $\mathbf{d}$ defined as the vector connecting them.

Strictly implementing UNDX requires constructing a full orthonormal basis matrix for the remaining dimensions, an $\mathcal{O}(n^3)$ operation computationally prohibitive for a high-frequency DE loop. To optimize performance, a randomized

Algorithm 4 Simplex Crossover (DE/SPX)

Require: Target vector $\mathbf{x}_1$, Donor vector $\mathbf{x}_2$, Random vector $\mathbf{x}_3$, Dimension D

Ensure: Trial vector $\mathbf{u}_i$

$m \leftarrow 3$ *// Number of parents*
$\epsilon \leftarrow \sqrt{m}$ *// Expansion rate for 3 parents*
// Calculate the center of mass
for $j = 1$ to D **do**
  $O_j \leftarrow (x_{1,j} + x_{2,j} + x_{3,j})/3.0$
end for
// Calculate the expanded vertices
for $j = 1$ to D **do**
  $y_{1,j} \leftarrow O_j + \epsilon \cdot (x_{1,j} - O_j)$
  $y_{2,j} \leftarrow O_j + \epsilon \cdot (x_{2,j} - O_j)$
  $y_{3,j} \leftarrow O_j + \epsilon \cdot (x_{3,j} - O_j)$
end for
// Generate two random weights and uniformly sample inside the triangle
$r_1 \leftarrow$ random uniform number in $[0, 1]$
$r_2 \leftarrow$ random uniform number in $[0, 1]$
if $r_1 + r_2 > 1.0$ **then**
  $r_1 \leftarrow 1.0 - r_1$
  $r_2 \leftarrow 1.0 - r_2$
end if
$w_1 \leftarrow 1.0 - r_1 - r_2$
$w_2 \leftarrow r_1$
$w_3 \leftarrow r_2$
// Assemble child vector using calculated weights
for $j = 1$ to D **do**
  $u_{i,j} \leftarrow w_1 \cdot y_{1,j} + w_2 \cdot y_{2,j} + w_3 \cdot y_{3,j}$
end for
return $\mathbf{u}_i$

Fig. 2. Geometric visualization of the UNDX parent vectors and the resulting normally distributed offspring cloud stretched along the primary axis $\mathbf{d}$ spanning between the Target (P1) and Donor (P2).

orthogonal projection shortcut was implemented. A single vector $\mathbf{z}$ consisting of standard Gaussian noise was generated, and its projection onto the primary axis $\mathbf{d}$ was subtracted to yield the purely orthogonal noise component $\mathbf{z}_{orth}$. The offspring vector was constructed as:

$$\mathbf{u}_i(t) = \mathbf{m} + \xi\mathbf{d} + \sigma_\eta D\mathbf{z}_{orth} \tag{2}$$

where primary variance $\xi \sim \mathcal{N}(0, 0.5^2)$, D is the orthogonal distance of the third parent to the primary axis, and orthogonal variance is scaled by $\sigma_\eta = 0.35/\sqrt{n_{dim}}$.

Algorithm 5 Unimodal Normal Distribution Crossover (DE/UNDX)

Require: Target vector $\mathbf{x}_1$, Donor vector $\mathbf{x}_2$, Random vector $\mathbf{x}_3$, Dimension D
Ensure: Trial vector $\mathbf{u}_i$

$\sigma_\xi \leftarrow 0.5$
$\sigma_\eta \leftarrow 0.35/\sqrt{D}$
// 1. Define the primary search axis
$\mathbf{m} \leftarrow \frac{1}{2}(\mathbf{x}_1 + \mathbf{x}_2)$
$\mathbf{d} \leftarrow \mathbf{x}_1 - \mathbf{x}_2$
// 2. Calculate orthogonal distance D_{orth} from $\mathbf{x}_3$
$\mathbf{v} \leftarrow \mathbf{x}_3 - \mathbf{x}_1$
$\mathbf{v}_{proj} \leftarrow \left(\frac{\mathbf{v}\cdot\mathbf{d}}{\mathbf{d}\cdot\mathbf{d}}\right)\mathbf{d}$ *// Project $\mathbf{v}$ onto the primary axis*
$\mathbf{v}_{orth} \leftarrow \mathbf{v} - \mathbf{v}_{proj}$
$D_{orth} \leftarrow \|\mathbf{v}_{orth}\|$ *// Magnitude of the orthogonal component*
// 3. Generate orthogonal Gaussian noise vector
$\mathbf{z} \leftarrow$ random vector of length D sampled from $\mathcal{N}(0, 1)$
$\mathbf{z}_{proj} \leftarrow \left(\frac{\mathbf{z}\cdot\mathbf{d}}{\mathbf{d}\cdot\mathbf{d}}\right)\mathbf{d}$
$\mathbf{z}_{orth} \leftarrow \mathbf{z} - \mathbf{z}_{proj}$
// 4. Assemble the trial vector
$\xi \leftarrow$ random number sampled from $\mathcal{N}(0, \sigma_\xi^2)$
$\mathbf{u}_i \leftarrow \mathbf{m} + \xi\mathbf{d} + \sigma_\eta \cdot D_{orth} \cdot \mathbf{z}_{orth}$
return $\mathbf{u}_i$

E. Parent-Centric Crossover (DE/PCX)

To evaluate an exploitative continuous operator, the Generalized PCX algorithm was integrated, as visualized in Figure 3. The collective center of mass $\mathbf{O}$ for the three parents was calculated, and the primary direction vector $\mathbf{d}_i$ was drawn from $\mathbf{O}$ strictly toward the primary parent (the target vector).

The average perpendicular distance $\bar{D}$ of the other two parents to this primary axis was computed. Using the identical orthogonal projection shortcut ($\mathbf{z}_{orth}$) developed for UNDX, a Gaussian scatter cloud was generated explicitly centered on the primary parent. The standard deviations for both the primary and orthogonal spreads were fixed at highly exploitative values ($\sigma_\xi = 0.1$, $\sigma_\eta = 0.1$).

F. Experimental Setup

To empirically evaluate the performance of these five operators, a diverse suite of eight 30-dimensional benchmark functions was compiled, representing macro-gradient, deceptive, and highly multi-modal topologies (Ackley, Griewank, Levy,

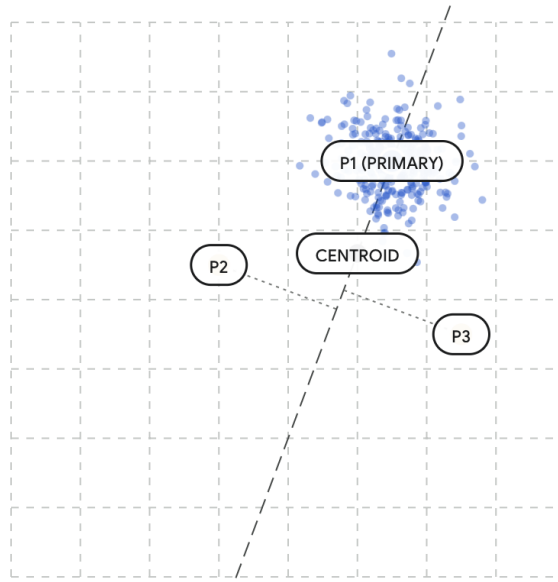

Fig. 3. Geometric visualization of the highly exploitative PCX offspring distribution clustered strictly around the Primary Parent (P1) towards the collective center of mass.

Algorithm 6 Parent-Centric Crossover (DE_PCX)

Require: Target vector $\mathbf{x}_1$ (Primary), Donor vector $\mathbf{x}_2$, Random vector $\mathbf{x}_3$, Dimension D
Ensure: Trial vector $\mathbf{u}_i$

$\sigma_\xi \leftarrow 0.1$ *// Highly exploitative primary spread*
$\sigma_\eta \leftarrow 0.1$ *// Highly exploitative orthogonal spread*
// 1. Calculate Center of Mass and Primary Axis
$\mathbf{O} \leftarrow \frac{1}{3}(\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3)$
$\mathbf{d} \leftarrow \mathbf{x}_1 - \mathbf{O}$ *// Direction vector points strictly toward Primary Parent*
// 2. Calculate average orthogonal distance $\bar{D}$ of other parents
$\mathbf{v}_2 \leftarrow \mathbf{x}_2 - \mathbf{x}_1$
$\mathbf{v}_{2,proj} \leftarrow \left(\frac{\mathbf{v}_2\cdot\mathbf{d}}{\mathbf{d}\cdot\mathbf{d}}\right)\mathbf{d}$
$D_2 \leftarrow \|\mathbf{v}_2 - \mathbf{v}_{2,proj}\|$
$\mathbf{v}_3 \leftarrow \mathbf{x}_3 - \mathbf{x}_1$
$\mathbf{v}_{3,proj} \leftarrow \left(\frac{\mathbf{v}_3\cdot\mathbf{d}}{\mathbf{d}\cdot\mathbf{d}}\right)\mathbf{d}$
$D_3 \leftarrow \|\mathbf{v}_3 - \mathbf{v}_{3,proj}\|$
$\bar{D} \leftarrow \frac{D_2+D_3}{2}$ *// Average perpendicular distance*
// 3. Generate orthogonal Gaussian noise vector
$\mathbf{z} \leftarrow$ random vector of length D sampled from $\mathcal{N}(0, 1)$
$\mathbf{z}_{proj} \leftarrow \left(\frac{\mathbf{z}\cdot\mathbf{d}}{\mathbf{d}\cdot\mathbf{d}}\right)\mathbf{d}$
$\mathbf{z}_{orth} \leftarrow \mathbf{z} - \mathbf{z}_{proj}$
// 4. Assemble the trial vector (Centered exactly on the Primary Parent)
$\xi \leftarrow$ random number sampled from $\mathcal{N}(0, \sigma_\xi^2)$
$\mathbf{u}_i \leftarrow \mathbf{x}_1 + \xi\mathbf{d} + \sigma_\eta \cdot \bar{D} \cdot \mathbf{z}_{orth}$
return $\mathbf{u}_i$

Salomon, Rastrigin, Schwefel, Michalewicz, and Alpine1). The DE parameters were held constant across all solvers with a population size of $NP = 100$, mutation factor $F = 0.5$, and $CR = 0.8$. Each solver was subjected to 30 independent runs per function, capped at a termination criteria of 3,000 iterations to ensure statistical robustness.

IV. Empirical Procedure

To evaluate the algorithms, the experimental setup was designed to stress test the Particle Swarm Optimisation variants against increasingly large search spaces to capture how algorithmic performance degenerates or improves with dimensionality.

A. Benchmark Functions

The capability of the algorithm variants was tested against five classic benchmarking optimisation functions, consisting of both unimodal and multi-modal landscapes. The functions were evaluated across dimensions (D) 100, 500, and 1000, and the specific character domains were as follows:

- **Sphere**: A smooth, unimodal, strongly convex landscape. Search domain: $[-100.0, 100.0]^D$.
- **Elliptic**: A highly ill-conditioned function, representing stretched topologies. Search domain: $[-100.0, 100.0]^D$.
- **Rastrigin**: A highly multi-modal function containing numerous periodic local minima. Search domain: $[-5.12, 5.12]^D$.
- **Rosenbrock**: A non-convex, narrow "valley" topology testing local minima avoidance. Search domain: $[-30.0, 30.0]^D$.
- **Schwefel 1.2**: A deeply non-separable unimodal function. Search domain: $[-100.0, 100.0]^D$.

B. Experimental Parameters

Fairness across all algorithms was strictly maintained. Every algorithm executed for exactly 30 independent runs to ensure robust averages. Each single run was strictly terminated after 5000 iterations. The swarm size was tightly constrained to 30 particles, the inertia weight (w) was set to 0.729844, and both the cognitive (c_1) and social (c_2) acceleration coefficients were defined equally as 1.49618. These specific parameters were selected and are based on the constriction factor of Clerc and Kennedy [1]. The parameters are proven to guarantee convergent trajectory behaviour and prevent systemic velocity explosion. Because standard baseline parameters across every algorithm were used, any performance discrepancies measured are attributed to the underlying algorithm architecture.

C. Evaluation Metrics

To determine the efficiency and accuracy of the underlying architectures, final fitness values across all benchmarking functions were logged for all 30 independent runs. The **Mean Error** and **Standard Deviation** are reported under tabular conditions in Table I.

Additionally, logarithmic convergence profiles plotting the generation sequence against the global best fitness evaluations are provided to visually represent optimisation speed and trajectory stagnation characteristics over time.

V. Research Results

This section presents the empirical findings of the study. First, convergence behaviours and structural biases are analysed. Following this, a specific note on the Rosenbrock anomaly is discussed.

A. Convergence Behaviours and Algorithmic Structural Bias

The convergence profiles for $D = 1000$ across the benchmark functions are presented in Figures 4 through 8. Across the Sphere, Elliptic, Rastrigin, and Schwefel 1.2 landscapes, a consistent performance hierarchy emerges.

The Canonical PSO consistently exhibited the poorest performance. Driven by independent, per-dimension random initialisation, the Canonical swarm suffers from severe velocity explosion in high-dimensional spaces, causing particles to rapidly expand the search radius of the particles and waste evaluations on out-of-bounds roaming.

The purely Stochastic Scaling PSO successfully dampens this velocity explosion through dimensional coupling, keeping the swarm stable. However, the stochastic scaling algorithm plateaus early in the search process. Because the initial positions of the stochastic scaling swarm are scattered uniformly at random, the coupled velocity updates restrict the reachable space of the swarm, preventing the swarm from navigating to the global optimum from a distant starting point.

The Subspace Initialisation PSO bypasses the initial velocity explosion by seeding the swarm entirely along a one-dimensional line passing through the center of the bounds. It is important to note the global optima of these standard benchmark functions lie exactly at the origin. Therefore, Subspace Initialisation effectively drops the swarm precisely on the target landscape at Iteration 0, providing a massive, head-start. However, because the algorithm subsequently reverts to standard Canonical PSO velocity updates, the subspace initialisation algorithm eventually succumbs to high-dimensional chaos and struggles to fine-tune the exploitation of the swarm.

The **Hybrid PSO** heavily dominates these four benchmarks, but the superiority of the hybrid model is not solely due to this initialisation advantage. While the hybrid PSO shares the exact same origin-drop advantage as the Subspace algorithm, the Hybrid immediately transitions into Stochastic Scaling for the iterative velocity updates. This mechanism actively controls particle momentum, preventing the swarm from exploding off the initial subspace line into chaotic jitter. This biased starting position with damped velocity mechanics results in a staircase convergence, allowing the swarm to converge significantly deeper into the local minima than pure Subspace Initialisation.

Fig. 4. Convergence Profile for Sphere ($D = 1000$)

Fig. 5. Convergence Profile for Elliptic ($D = 1000$)

Fig. 6. Convergence Profile for Schwefel 1.2 ($D = 1000$)

TABLE I
MEAN BEST FITNESS AND STANDARD DEVIATION OVER 30 INDEPENDENT RUNS

Function	D	Canonical PSO	Stochastic Scaling	Subspace Init	Hybrid PSO
Sphere	100	1.43e+04 ± 8.98e+03	3.33e+02 ± 1.83e+03	7.55e-08 ± 2.93e-07	**2.73e-11 ± 1.44e-10**
	500	2.82e+05 ± 3.86e+04	2.23e+04 ± 7.48e+03	1.81e+01 ± 3.19e+01	**4.40e-01 ± 7.48e-01**
	1000	1.02e+06 ± 6.98e+04	1.56e+05 ± 3.11e+04	5.34e+01 ± 9.31e+01	**4.98e+00 ± 6.43e+00**
Elliptic	100	8.59e+08 ± 5.15e+08	1.99e+07 ± 6.11e+07	2.54e-04 ± 9.82e-04	**1.38e-09 ± 4.91e-09**
	500	1.31e+10 ± 3.99e+09	3.78e+08 ± 1.89e+08	2.89e+05 ± 8.77e+05	**4.94e+03 ± 8.96e+03**
	1000	3.29e+10 ± 6.32e+09	2.89e+09 ± 5.75e+08	1.58e+06 ± 2.37e+06	**2.52e+05 ± 4.14e+05**
Rosenbrock	100	2.40e+07 ± 4.28e+07	2.01e+02 ± 5.67e+01	8.89e+01 ± 1.93e+00	**8.73e+01 ± 9.90e-01**
	500	1.23e+09 ± 2.69e+08	7.65e+06 ± 3.06e+06	6.04e+02 ± 2.87e+02	**4.99e+02 ± 1.16e+01**
	1000	4.44e+09 ± 4.63e+08	8.52e+07 ± 3.33e+07	1.92e+03 ± 2.06e+03	**1.03e+03 ± 8.94e+01**
Rastrigin	100	6.21e+02 ± 8.48e+01	1.88e+02 ± 3.67e+01	6.30e-01 ± 2.42e+00	**8.69e-09 ± 2.45e-08**
	500	3.96e+03 ± 1.64e+02	1.58e+03 ± 1.46e+02	1.02e+01 ± 2.36e+01	**3.06e-01 ± 4.75e-01**
	1000	9.49e+03 ± 2.30e+02	5.10e+03 ± 2.66e+02	2.77e+01 ± 3.79e+01	**6.20e+00 ± 1.97e+01**
Schwefel12	100	8.93e+04 ± 4.32e+04	1.02e+04 ± 1.38e+04	3.94e+00 ± 6.35e+00	**1.65e-03 ± 7.03e-03**
	500	1.77e+06 ± 3.79e+05	7.22e+05 ± 2.69e+05	1.77e+03 ± 3.86e+03	**7.61e+01 ± 1.70e+02**
	1000	6.89e+06 ± 9.79e+05	3.07e+06 ± 9.26e+05	7.73e+03 ± 2.47e+04	**2.67e+02 ± 4.08e+02**

Fig. 7. Convergence Profile for Rastrigin ($D = 1000$)

Note on the Rosenbrock Anomaly

While the Hybrid PSO dominates symmetrically centered topologies, the algorithm immediately stagnates at roughly 10^3 on the Rosenbrock benchmark as illustrated in Figure 8. This stagnation occurs due to how the algorithm interacts with the notoriously flat parabolic valley of the Rosenbrock function. Subspace Initialisation forces the swarm to start at the origin $[0, \ldots, 0]$ with zero initial velocity. Trapped on this flat valley floor without initial momentum, and heavily restricted by the dampening of Stochastic Scaling, the particles cannot generate the velocity required to navigate toward the global minimum at $[1, \ldots, 1]$. This anomaly highlights a critical trade-off: mechanisms to prevent velocity explosions on steep topologies actively hinder exploration on flat-valley landscapes.

Fig. 8. Convergence Profile for Rosenbrock ($D = 1000$)

VI. CONCLUSION

The primary objective of this study was to evaluate strategies for mitigating the curse of dimensionality in large scale optimisation problems (LSOPs). A hybrid particle swarm optimisation (HPSO) algorithm was developed, integrating Subspace Initialisation and Stochastic Scaling. The performance of this hybrid was benchmarked against Canonical PSO and the independent constituent algorithms of the hybrid across five landscapes at dimensionalities up to $D = 1000$.

The empirical results supported the initial hypothesis for the majority of the tested landscapes. By combining the geometrically constrained, zero velocity starting conditions of Subspace Initialisation with the coupled velocity dampening of Stochastic Scaling, the Hybrid PSO successfully prevented high-dimensional velocity explosion and excessive particle roaming. The Hybrid PSO outperformed the Canonical, purely Stochastic, and purely Subspace swarms on the Sphere, Elliptic, Rastrigin, and Schwefel 1.2 benchmarks, demonstrating stable convergence.

However, this investigation also identified a critical algorithmic limitation through the Rosenbrock anomaly. The complete stagnation of the Hybrid swarm on the Rosenbrock function proved that while eliminating initial momentum is highly beneficial for symmetrically steep functions, this elimination is actively detrimental on flat-valley landscapes where initial momentum is strictly required for traversal.

Conducting this research highlighted that algorithmic performance in LSOPs cannot be evaluated purely on final fitness metrics. An understanding of how the mechanics of an algorithm interact with the specific spatial landscape of a problem is also important.

Future work should address the initialisation bias present in this study by benchmarking the Hybrid algorithm against "shifted" landscape variants, where the global optimum is intentionally displaced from the geometric center of the search space. Future studies should investigate adaptive initialisation strategies to detect flat-valley landscapes in order to inject necessary initial momentum to address stagnation vulnerabilities observed in the Rosenbrock landscape.

APPENDIX A
ACRONYMS

- HPSO: Hybrid Particle Swarm Optimisation
- LSOP: Large Scale Optimisation Problem
- PSO: Particle Swarm Optimisation

APPENDIX B
SYMBOLS

- c_1: Cognitive acceleration coefficient
- c_2: Social acceleration coefficient
- C: Geometric center of the search space bounds
- d: Orthonormal seed vector
- D: Dimensionality of the search space

- $f(x)$: Objective fitness function evaluated at point x
- $Gbest$ / $\hat{y}$: Global best position of the swarm
- k: Number of stochastic scaling groups
- N: Total number of particles in the swarm
- $Pbest_i$ / y_i: Personal best position of particle i
- r_1, r_2: Stochastic random vectors drawn from $U(0, 1)$
- t: Current iteration number
- V_i / v_i: Velocity vector of particle i
- w: Inertia weight
- X_i / x_i: Position vector of particle i

References

[1] J. Kennedy and R. C. Eberhart, "Particle swarm optimization," in *Proceedings of the International Conference on Neural Networks*, 1995, pp. 1942–1948.

[2] E. T. Oldewage, C. W. Cleghorn, and A. P. Engelbrecht, "Solving large scale optimisation problems using particle swarm optimisation," in *Proceedings of the IEEE Symposium Series on Computational Intelligence*, 2019, pp. 1–8.

[3] E. T. Van Zyl and A. P. Engelbrecht, "A subspace-based method for PSO initialization," in *Proceedings of the IEEE Symposium Series on Computational Intelligence*, 2015, pp. 1877–1883.